Qually: An Accessible LLM Audit Tool for Social Science Research

Kartik Trivedi
University of New Hampshire
hello@kartiktrivedi.in

November 5, 2025

Abstract

Large Language Models (LLMs) are increasingly used in social science research, yet systematic evaluation and auditing of these models remains challenging for researchers without programming expertise. We present Qually, a graphical user interface (GUI)-based tool designed to democratize LLM auditing for social science researchers. Qually provides a unified interface for conducting experiments across multiple LLM providers (OpenAI, Anthropic, Google, Mistral, Grok, and DeepSeek), enabling systematic comparison of model behaviors, prompt engineering studies, and bias evaluations. The tool features experiment management capabilities, secure API key handling using Fernet encryption, built-in rate limiting with provider-specific strategies, and structured data export for reproducible research. Qually addresses the gap between powerful but technically complex LLM evaluation frameworks and the needs of social science researchers who require accessible, reproducible methods for model auditing. This paper describes the system architecture, features, and use cases. A comprehensive user manual is provided as a separate document with detailed step-by-step instructions for researchers.

1 Introduction

1.1 Context and Motivation

Large Language Models (LLMs) have become integral to social science research, enabling applications ranging from content analysis to survey generation and experimental manipulations [1]. Social scientists increasingly rely on LLMs for tasks such as generating survey questions, analyzing qualitative data, creating experimental stimuli, and simulating human behavior. However, systematic evaluation and auditing of LLM behavior remains a significant challenge, particularly for researchers without extensive programming experience.

The increasing adoption of LLMs in social science research creates a pressing need for accessible tools that enable researchers to systematically evaluate and compare model behaviors. Without such tools, researchers may rely on ad-hoc evaluations that lack reproducibility or may be unable to conduct systematic audits at all.

1.2 The Problem

Existing LLM evaluation frameworks, while powerful, typically require:

- **Programming expertise**: Most evaluation tools require Python programming knowledge
- **Command-line interface familiarity**: Many tools operate through terminal interfaces
- **Complex configuration management**: Setting up experiments requires editing configuration files or writing code

- **Manual experiment tracking**: Researchers must manually track experimental conditions, parameters, and results
- **Provider-specific implementations**: Different providers require different API calls and parameter handling

These barriers prevent many social science researchers from conducting systematic LLM audits, limiting the reproducibility and rigor of LLM-based research in the social sciences.

1.3 Our Contribution

Qually addresses these challenges by providing:

- **GUI-based interface** requiring no programming knowledge
- **Unified multi-provider support** with a single interface for all providers
- **Integrated experiment management** with automatic tracking of conditions and parameters
- **Reproducible research workflows** with structured data export and experiment definitions
- **Secure API key management** using industry-standard encryption
- **Built-in rate limiting** to prevent API quota issues

Qually version 2.0.0 (LE2) is freely available at `https://github.com/kartik0trivedi/Qually` and supports macOS and Windows platforms.

2 Related Work

2.1 LLM Evaluation Frameworks

Several frameworks exist for evaluating LLMs, including HELM [1], BIG-bench [1], and others. These frameworks typically focus on comprehensive benchmark evaluation across multiple tasks and domains. However, they are designed primarily for technical users and require substantial programming expertise to use effectively.

Most existing evaluation tools operate through command-line interfaces or require extensive code configuration. While powerful, these tools create barriers for researchers who may be experts in their domain but lack extensive programming experience.

2.2 LLM Auditing in Social Science

Social science research has increasingly focused on auditing LLMs for bias, fairness, and behavioral consistency. Research has examined gender bias, racial bias, and other forms of bias in LLM outputs. However, the tools used for these audits typically require programming expertise, limiting who can conduct such evaluations.

The need for accessible auditing tools in social science is particularly acute given the increasing use of LLMs in research contexts where bias and fairness are critical concerns.

2.3 Gap Analysis

Existing tools lack several key features needed by social science researchers:

- **Accessibility**: Most tools require programming knowledge
- **Multi-provider comparison**: Few tools enable easy comparison across multiple providers
- **Experiment management**: Limited support for managing multiple conditions and experiments
- **Reproducibility**: Tools often lack structured export formats for reproducible research
- **Security**: API key management is often handled manually without encryption

Qually addresses these gaps by providing a GUI-based tool specifically designed for social science researchers conducting systematic LLM audits.

3 System Architecture and Design

3.1 Overall Architecture

Qually is built on a modular architecture (Figure 1) consisting of five main layers:

- **GUI Layer**: PyQt6-based user interface providing tab-based organization
- **Experiment Manager**: Core experiment logic, execution, and result tracking
- **Provider Abstraction Layer**: Unified interface for multiple LLM providers using factory pattern
- **Data Management**: CSV import/export and prompt management with JSON storage
- **Security Layer**: Encrypted API key storage using Fernet symmetric encryption

The architecture follows a separation of concerns principle, with each layer handling distinct responsibilities. The GUI layer handles user interaction, the Experiment Manager coordinates experiment execution, the Provider Abstraction Layer provides a unified interface to different LLM APIs, and the Security Layer ensures sensitive data is protected.

3.2 Core Components

3.2.1 Experiment Manager

The ExperimentManager class serves as the central coordinator for all experimental activities. It manages:

- **Experiment creation**: Each experiment receives a unique timestamp-based ID and stores metadata including name, description, and creation timestamp
- **Condition management**: Conditions define specific LLM configurations (provider, model, parameters) to test
- **Experiment execution**: Coordinates running prompts across multiple conditions with progress tracking
- **Result storage**: Saves results in structured JSON and CSV formats with timestamps

The Experiment Manager uses a project-based organization system, where each project folder contains experiment definitions (`experiments.json`), prompts, results, and encrypted API keys. This structure enables easy backup, sharing, and version control of research projects.

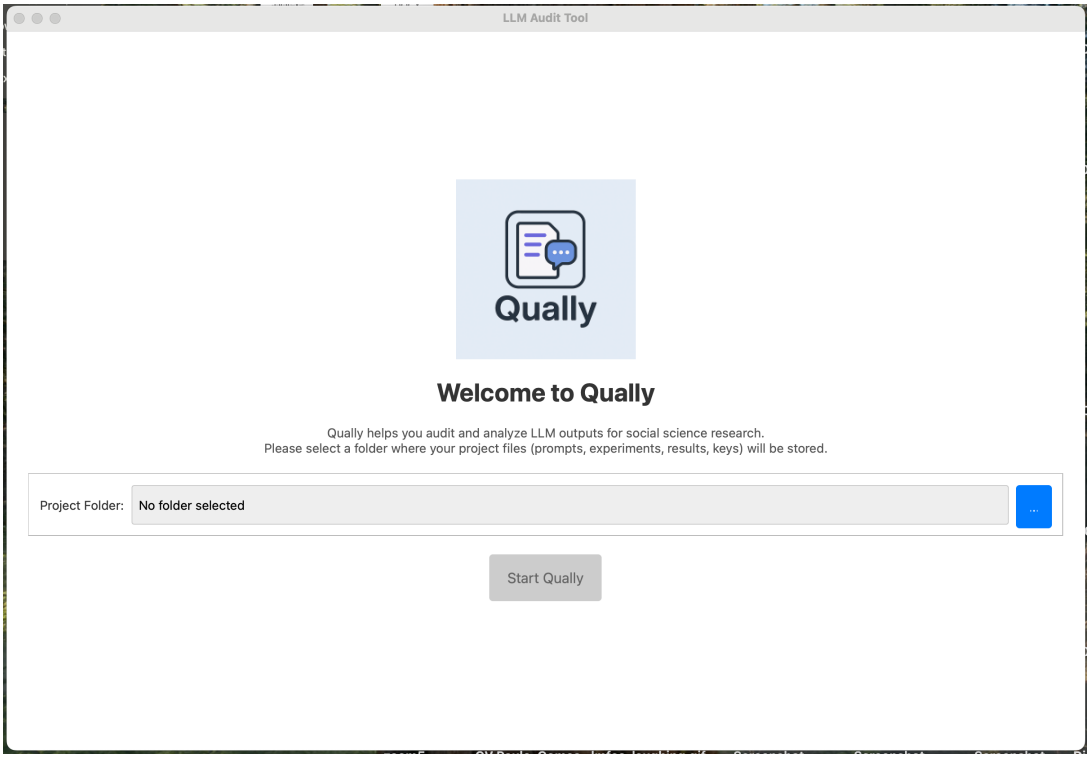

Figure 1: Qually Main Interface: Tab-based organization showing API Keys, Data Files, Prompts, Experiments, and Results tabs.

3.2.2 Provider Abstraction Layer

The Provider Abstraction Layer implements a factory pattern to create provider instances dynamically. Each provider (OpenAI, Anthropic, Google, Mistral, Grok, DeepSeek) inherits from a base `LLMProvider` class that defines a common interface. This abstraction allows researchers to specify experiments in terms of providers and models without needing to know provider-specific API details.

The unified interface includes:

- Standardized `generate()` method that accepts prompt, system prompt, and parameters
- Consistent error handling across all providers
- Unified response format with text, model, provider, and metadata fields
- Automatic parameter mapping from generic names (e.g., "temperature") to provider-specific parameters

3.2.3 API Key Management

The `APIKeyManager` class handles secure storage and retrieval of API keys using Fernet encryption (symmetric encryption from the cryptography library). Key features include:

- **Automatic key generation**: Each project generates a unique encryption key on first use
- **Encrypted storage**: API keys are encrypted using Fernet before being stored in `api_keys.json`
- **Key isolation**: Each project has its own encryption key, preventing cross-project key access

- **Error handling**: Graceful handling of corrupted or missing encryption keys

The encryption key is stored separately in `encryption_key.key`, ensuring that even if the encrypted API keys are accessed, they cannot be decrypted without the key file. This two-file approach provides defense in depth for sensitive credentials.

3.2.4 Rate Limiting System

The rate limiting system prevents hitting API quotas by implementing configurable delays between requests. Each provider has default rate limits (Table 1), which can be customized in `settings.json`.

The rate limiting implementation includes:

- **Provider-specific delays**: Different delays for different providers based on their rate limits
- **Thread-safe locking**: Uses threading locks to prevent race conditions in multi-threaded execution
- **Automatic retries**: Implements exponential backoff for HTTP 429 (rate limit) errors
- **Random jitter**: Adds small random delays to prevent thundering herd problems
- **Configurable parameters**: Rates can be adjusted via `settings.json` without code changes

The system uses the `requests` library's Retry adapter with automatic retries for rate limit errors (HTTP 429) and server errors (5xx), ensuring robust handling of transient API issues.

4 Features and Capabilities

4.1 Multi-Provider Support

Qually supports six major LLM providers, enabling researchers to compare models across different ecosystems. Each provider is implemented as a separate class inheriting from the base `LLMProvider` class, ensuring consistent behavior while accommodating provider-specific requirements.

Table 1 shows supported providers, example models, and default rate limits. The unified interface means researchers can switch between providers without changing their experimental workflow.

Provider	Example Models	Default Rate Limit
OpenAI	gpt-4, gpt-3.5-turbo, gpt-4-turbo	1.0 req/s
Anthropic	claude-3-opus, claude-3.5-sonnet, claude-3-haiku	0.5 req/s
Google	gemini-1.5-pro, gemini-1.5-flash	0.67 req/s
Mistral	mistral-large-latest, open-mixtral-8x7b	1.0 req/s
Grok (xAI)	grok-1	1.0 req/s
DeepSeek	deepseek-chat, deepseek-coder	1.0 req/s

Table 1: Supported LLM Providers and Default Rate Limits

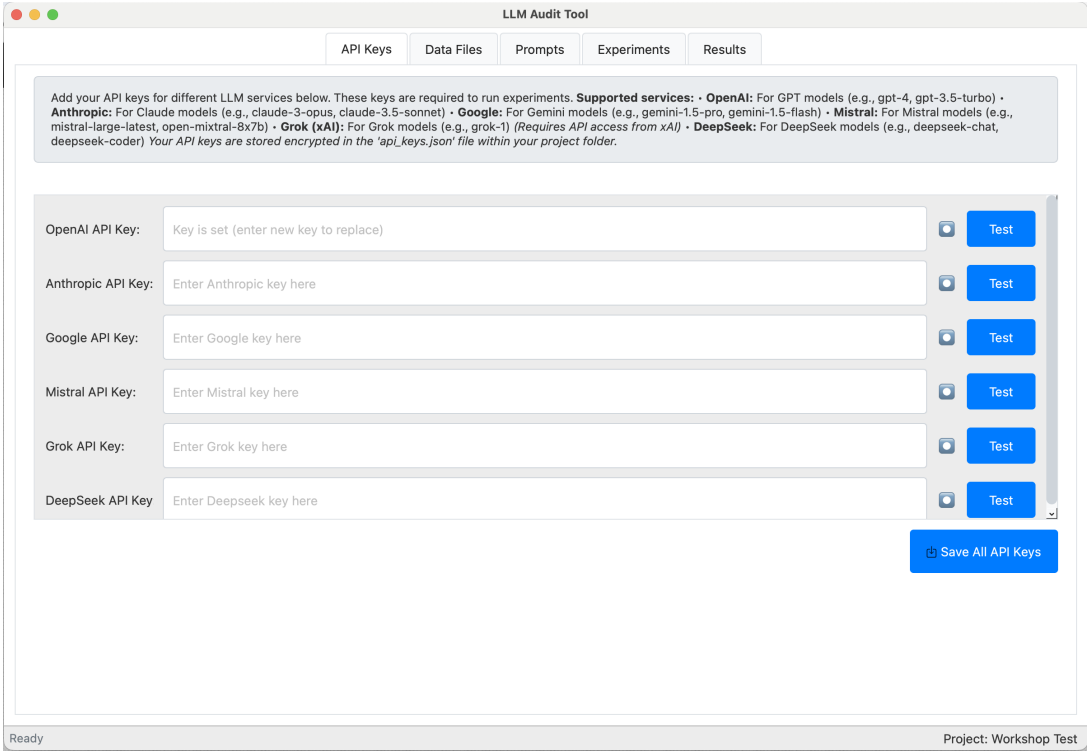

Figure 2: API Keys Configuration: Secure API key management interface with encrypted storage for multiple providers.

4.2 Experiment Management

The experiment management system enables researchers to:

- **Create experiments**: Define experiments with names, descriptions, and multiple conditions
- **Add conditions**: Specify different LLM configurations (provider, model, parameters) within a single experiment
- **Select prompts**: Choose which prompts to test in each experiment
- **Run experiments**: Execute experiments with progress tracking and logging
- **View results**: Examine results in tabular format with filtering and sorting

Each experiment is assigned a unique ID based on timestamp (`exp_<timestamp>`), and conditions receive IDs based on timestamp and sequence (`cond_<timestamp>_<sequence>`). This ensures unique identification while maintaining human-readable timestamps.

4.3 Data Management

Qually supports importing prompts and data via CSV files, enabling researchers to work with existing datasets. The data management system includes:

- **CSV import**: Import prompts and metadata from CSV files with automatic column detection
- **Prompt management**: Create, edit, and manage prompts with tags and categories

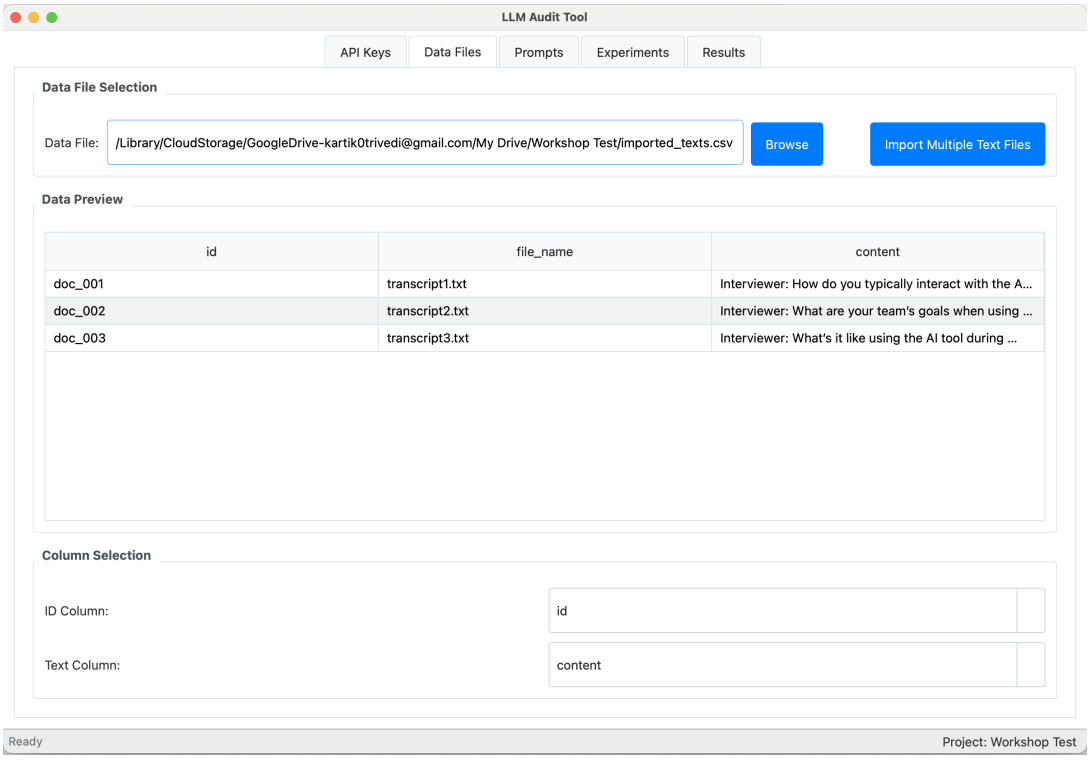

Figure 3: Experiment Creation: Interface for creating experiments with multiple conditions, allowing systematic comparison of different LLM configurations.

- **Structured storage**: Prompts stored in CSV format with support for system prompts, tags, and metadata
- **Results export**: Export results in CSV format with all experiment metadata

The prompt management system supports combining prepend text, data text, and prompt text, enabling researchers to create prompts from structured data. This is particularly useful for experiments where prompts need to be generated from datasets.

4.4 User Interface

The user interface is organized into five main tabs:

1. **API Keys**: Secure API key management with encryption
2. **Data Files**: CSV import and data file management
3. **Prompts**: Prompt creation, editing, and management
4. **Experiments**: Experiment creation and execution
5. **Results**: Results viewing, filtering, and export

The interface includes:

- **Progress tracking**: Progress bars and log messages during experiment execution
- **Error handling**: User-friendly error messages and validation
- **Status bar**: Shows current project and application status
- **Responsive design**: Interface remains responsive during long-running experiments

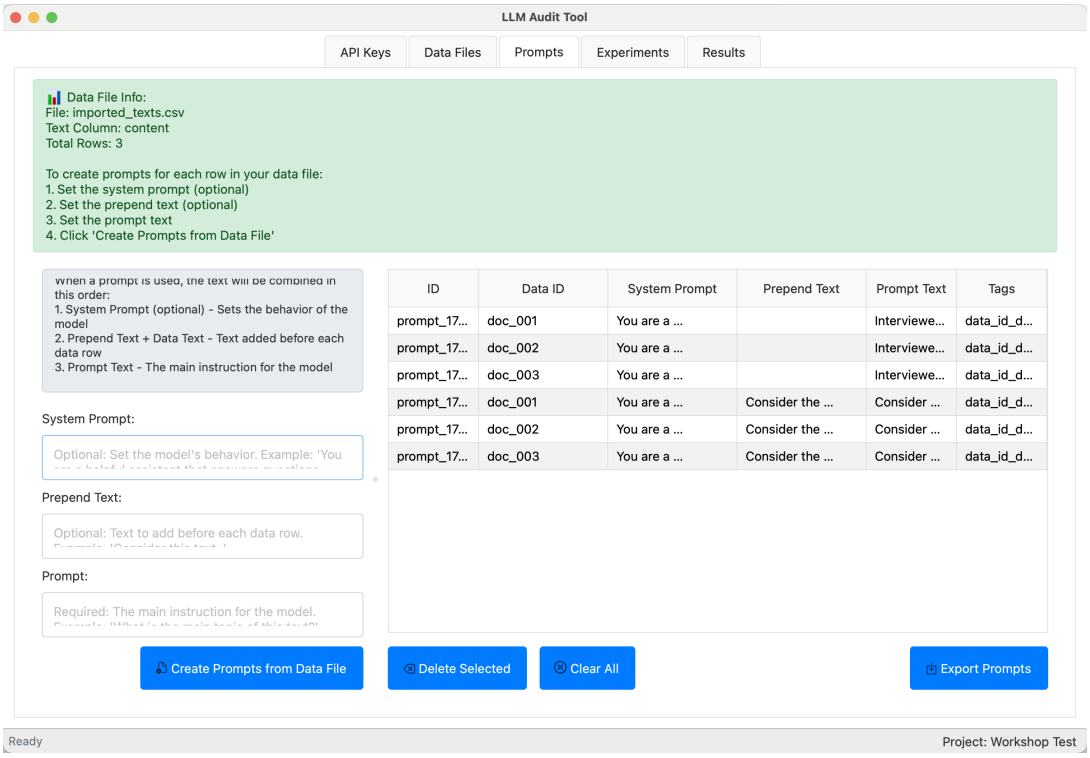

Figure 4: Prompt Management: Interface for importing and managing prompts, supporting CSV import and individual prompt creation.

5 Use Cases and Examples

5.1 Use Case 1: Bias Evaluation

A researcher wants to evaluate gender bias in LLM responses to job-related prompts. Using Qually, they:

1. Import a CSV file containing prompts with gender-neutral job descriptions
2. Create an experiment with three conditions:
   - Condition 1: GPT-4 with temperature=0.7
   - Condition 2: Claude-3.5-Sonnet with temperature=0.7
   - Condition 3: Gemini-1.5-Pro with temperature=0.7
3. Run the experiment and export results to CSV
4. Analyze responses for gender bias indicators

This workflow enables systematic comparison of bias across providers using identical prompts, ensuring fair comparison.

5.2 Use Case 2: Prompt Engineering Study

A researcher wants to test different prompt formulations to optimize LLM performance on a specific task. They:

1. Create multiple prompt variations (e.g., with/without examples, different instructions)

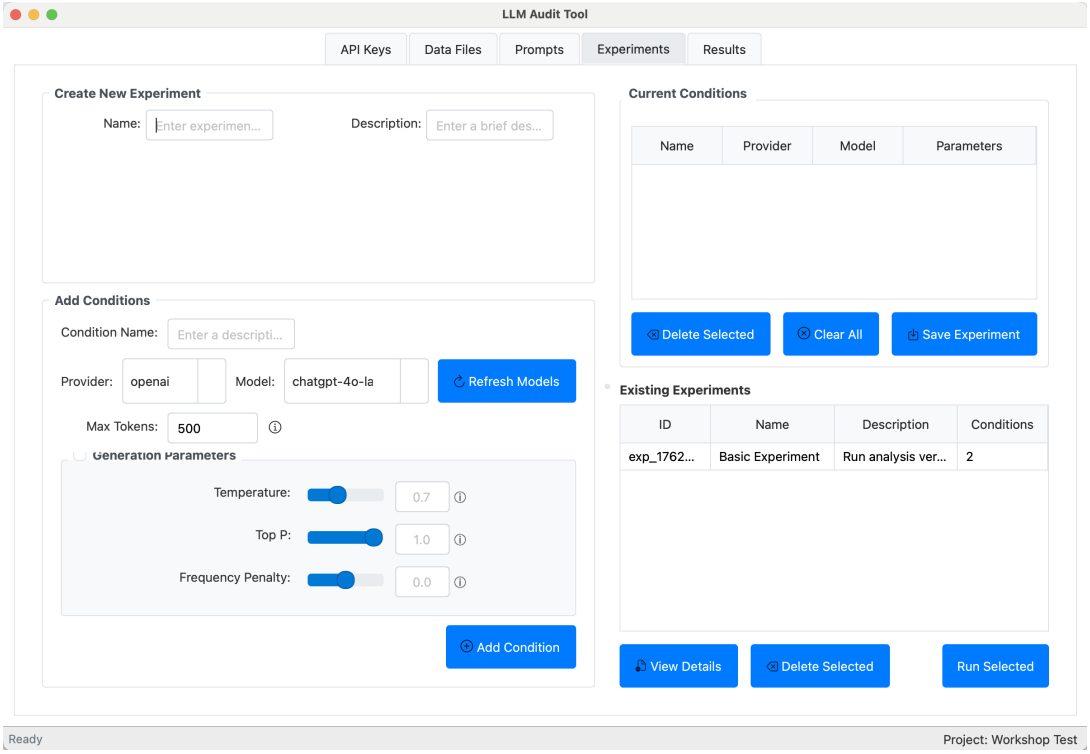

Figure 5: Results View: Table interface for viewing, filtering, and exporting experimental results with support for multiple experiments and conditions.

2. Set up an experiment with multiple conditions varying prompt structure
3. Run the experiment across multiple models
4. Compare performance metrics across prompt formulations

Qually's condition system allows testing multiple prompt variations and model configurations in a single experiment, streamlining prompt engineering workflows.

5.3 Use Case 3: Model Comparison

A researcher wants to compare responses from different models to the same set of prompts for a systematic evaluation. They:

1. Create an experiment with conditions for each model they want to compare
2. Select all prompts to test
3. Run the experiment and monitor progress
4. Export results for statistical analysis

This use case demonstrates Qually's strength in enabling systematic, reproducible comparisons across multiple models and providers.

6 Technical Details

6.1 Implementation

Qually is implemented in Python 3.11+ using the following key technologies:

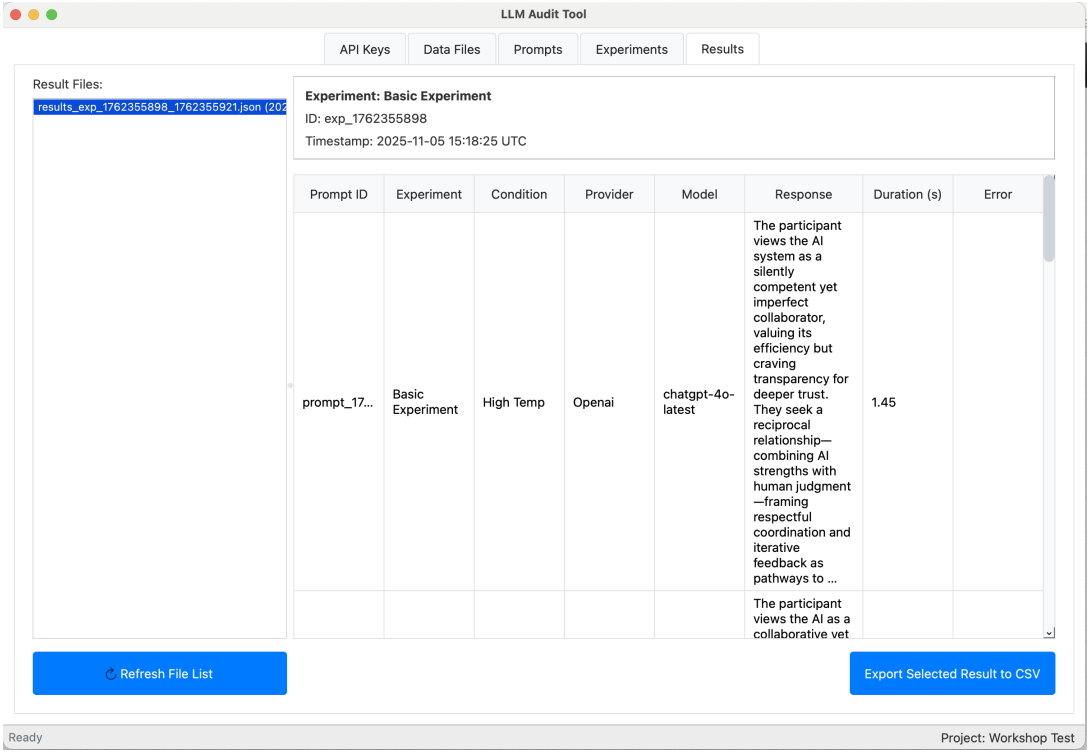

Figure 6: Experiment Execution: Real-time progress tracking during experiment execution with log messages and progress indicators.

- **PyQt6** (≥6.9.0): GUI framework for cross-platform desktop application
- **pandas** (≥2.2.0): Data manipulation for CSV import/export
- **requests** (≥2.32.0): HTTP library for API calls with retry support
- **cryptography** (≥44.0.0): Encryption library for API key security

The application is structured as a single-file GUI (`qually_gui.py`) and core logic module (`qually_tool.py`), enabling easy distribution and maintenance. The codebase follows object-oriented design principles with clear separation between GUI logic and business logic.

Qually supports both development mode (running directly from source) and packaged applications for macOS (using py2app) and Windows (using PyInstaller). The project structure is designed to be self-contained, with all project data stored in user-specified project folders.

6.2 Rate Limiting

The rate limiting system implements provider-specific delays to prevent hitting API quotas. Default delays are conservative and based on typical API limits:

- **OpenAI**: 1.0 second delay (60 requests/minute limit)
- **Anthropic**: 2.0 second delay (more conservative for Claude API)
- **Google**: 1.5 second delay (40 requests/minute limit)
- **Other providers**: 1.0 second delay

The system uses threading locks to ensure thread-safe rate limiting, and implements exponential backoff with jitter for handling rate limit errors. Rate limits can be configured via `settings.json` without modifying code.

The retry mechanism uses the `urllib3` Retry adapter with:

- Maximum 3 retries
- Exponential backoff (0.5s, 1s, 2s)
- Retry on HTTP 429 (rate limit) and 5xx (server errors)
- Random jitter to prevent synchronized retries

6.3 Security

API key security is implemented using Fernet symmetric encryption from the Python cryptography library. Fernet is a high-level interface built on top of AES-128 in CBC mode with HMAC-SHA256 for authentication.

Key security features:

- **Automatic key generation**: Each project generates a unique 256-bit encryption key
- **Separate key storage**: Encryption key stored in separate file (`encryption_key.key`)
- **Encrypted storage**: API keys encrypted before storage in `api_keys.json`
- **Key isolation**: Each project has its own encryption key
- **No key transmission**: Keys never transmitted over network

The encryption key is generated using `cryptography.fernet.Fernet.generate_key()`, which uses `os.urandom()` for cryptographically secure random number generation. The key is stored in binary format and never logged or displayed.

API keys are only decrypted in memory when needed for API calls, and are never stored in plaintext. This ensures that even if project files are accessed, API keys remain protected unless the attacker has both the encrypted keys file and the encryption key file.

7 User Manual and Documentation

A comprehensive user manual for Qually is available as a separate document. The user manual provides detailed, step-by-step instructions for:

- Installation and setup procedures
- Creating and managing projects
- Adding and securing API keys
- Importing data and creating prompts
- Setting up and running experiments
- Managing multiple conditions and providers
- Viewing and exporting results
- Troubleshooting common issues

- Best practices for reproducible research

The user manual is available at: `https://github.com/kartik0trivedi/Qually` (see `documentation/quall`
For quick reference, basic installation steps are:

1. Clone the repository: `git clone https://github.com/kartik0trivedi/Qually.git`
2. Install dependencies: `pip install -r requirements.txt`
3. Run the application: `python qually_gui.py`

Detailed usage instructions, troubleshooting guides, and examples are provided in the comprehensive user manual.

8 Limitations and Future Work

8.1 Current Limitations

Qually has several current limitations:

- **Provider-specific differences**: Different providers have different parameter names and ranges, which may require manual adjustment
- **Rate limiting trade-offs**: Conservative rate limits may slow down large experiments, but can be adjusted in settings
- **GUI limitations**: Advanced features may require understanding the underlying data structures
- **Single-threaded execution**: Experiments run sequentially, which may be slow for very large experiments
- **No built-in analysis**: Results export is provided, but statistical analysis must be done in external tools

8.2 Future Enhancements

Planned future enhancements include:

- **Additional providers**: Support for additional LLM providers (e.g., Cohere, Together AI)
- **Advanced analysis**: Built-in statistical analysis and visualization tools
- **Batch processing**: Parallel execution of experiments across multiple threads
- **Template system**: Pre-built experiment templates for common use cases
- **Integration with analysis tools**: Direct integration with R and Python analysis pipelines
- **Cloud sync**: Optional cloud synchronization for project backup
- **Advanced filtering**: More sophisticated result filtering and search capabilities

9 Discussion and Implications

9.1 Impact on Social Science Research

Qually has the potential to significantly impact social science research by:

- **Democratizing LLM evaluation**: Making systematic LLM auditing accessible to researchers without programming expertise
- **Enabling systematic audits**: Providing tools for rigorous, reproducible evaluation of LLM behavior
- **Supporting reproducibility**: Structured data export and experiment tracking enable reproducible research
- **Facilitating comparison**: Enabling easy comparison of models across different providers

By removing technical barriers, Qually enables more researchers to conduct systematic LLM evaluations, potentially improving the quality and rigor of LLM-based research in the social sciences.

9.2 Methodological Considerations

When using Qually for research, several methodological considerations are important:

- **Experimental design**: Researchers should carefully design experiments to ensure fair comparison across conditions
- **Parameter consistency**: When comparing models, ensure parameters are comparable across providers
- **Reproducibility**: Document all experimental conditions, including model versions and parameters
- **Sampling**: Consider prompt sampling strategies for large-scale evaluations
- **Analysis**: Use appropriate statistical methods for comparing results across conditions

Qually's structured export format facilitates these methodological considerations by ensuring all experimental details are captured and can be reviewed.

10 Conclusion

We have presented Qually, a GUI-based tool for conducting systematic LLM audits in social science research. Qually addresses the gap between powerful but technically complex LLM evaluation frameworks and the needs of social science researchers by providing an accessible, user-friendly interface for conducting experiments across multiple LLM providers.

Key contributions include:

- First GUI-based tool specifically designed for LLM auditing in social science
- Unified interface for multiple LLM providers
- Integrated experiment management with automatic tracking
- Secure API key management with encryption

- Built-in rate limiting to prevent API quota issues
- Structured data export for reproducible research

Qually enables researchers without programming expertise to conduct systematic LLM evaluations, potentially improving the quality and accessibility of LLM-based research in the social sciences. The tool is freely available and open-source, enabling the research community to contribute improvements and extensions.

We encourage researchers to use Qually for their LLM auditing needs and welcome feedback and contributions to improve the tool.

Acknowledgments

Funding for this application was provided by the Advanced Rehabilitation Research and Training (ARRT) Program on Employment at the University of New Hampshire, which is funded by the National Institute for Disability, Independent Living, and Rehabilitation Research, in the Administration for Community Living, at the U.S. Department of Health and Human Services (DHHS) under grant number 90AREM000401. The contents do not necessarily represent the policy of DHHS and you should not assume endorsement by the federal government (EDGAR, 75.620 (b)).

References

[1] Qually GitHub Repository. `https://github.com/kartik0trivedi/Qually`